Capítulo 5

Subprogramas: Funciones y Procedimientos

1. Programación estructurada

La Programación estructurada es un paradigma de programación[1] donde se enfatiza el orden y agrupamiento de operaciones a realizar.
Pascal es un lenguaje que responde plenamente a este estilo de programación. El paradigma estructurado tiene las siguientes características[2] :

a) El programa completo tiene un diseño modular.
b) Los módulos se diseñan con metodología descendente también denominada "top-down. (ya explicada en el Capítulo 2).
c) Cada módulo se codifica utilizando las tres estructuras de básicas de control (secuenciales, selectivas y repetitivas)

1.1 Diseño modular

Cuando en la actualidad se habla de Programación estructurada, nos solemos referir a la división de un programa en partes más manejables (usualmente denominadas segmentos o módulos). Una regla práctica para lograr este propósito es establecer que cada segmento del programa no exceda, en longitud, de una página de codificación, o sea, alrededor de 50 líneas.

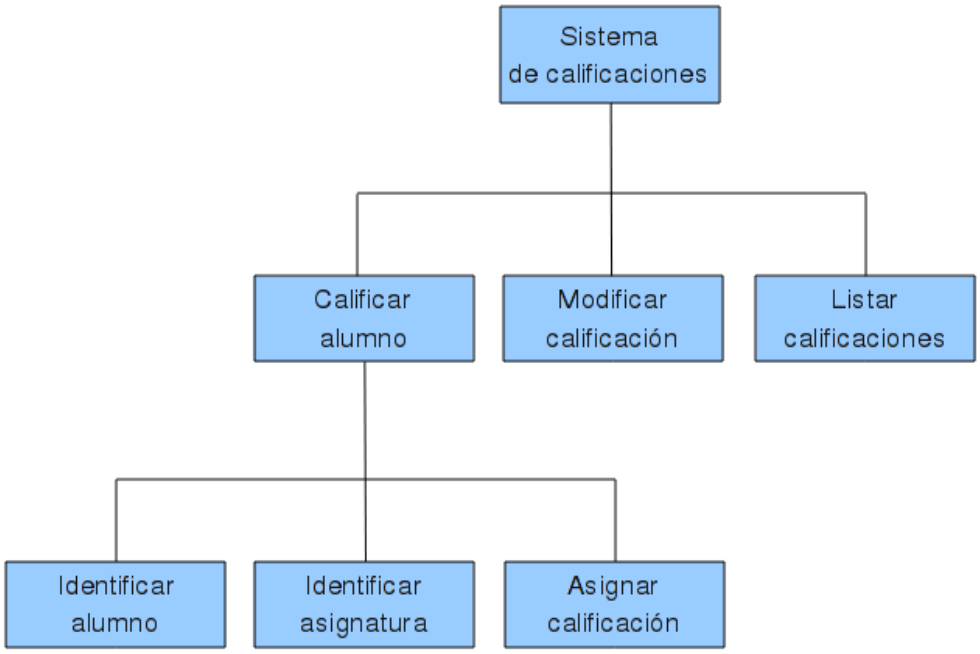
Así, la visión moderna de un programa estructurado es un compuesto de segmentos, los cuales puedan estar constituidos por unas pocas instrucciones o por una página o más de código.

Esta técnica de programación conlleva las siguientes ventajas:
a) El costo de resolver varios subproblemas de forma aislada es con frecuencia menor que el de abordar el problema global.
b) Facilita el trabajo simultáneo en paralelo de distintos grupos de programadores.
c) Posibilita en mayor grado la reutilización del código (especialmente de alguno de los módulos) en futuras aplicaciones.

1.2 Estructuras básicas de control

Hemos visto, en el capítulo anterior, programas Pascal elementales compuestos de sentencias simples. Las sentencias en estos programas aparecen en **secuencia** y estas son ejecutadas en el orden en que han sido escritas, de la primera hasta la última. Esta es la estructura de control natural de ejecución de programa. Este orden puede ser alterado y controlado por las estructuras de **decisión** y **repetición.** Más adelante se estudiará estas dos últimas estructuras básicas de control.

[1]Los paradigmas de programación nos indican las diversas formas que, a lo largo de la evolución de los lenguajes, han sido aceptadas como estilos para programar y para resolver los problemas por medio de una computadora.
[2]Análisis y Diseño de Algoritmos . López Gustavo, Jeder Ismael y Vega Augusto. Editorial Alfaomega. 2009

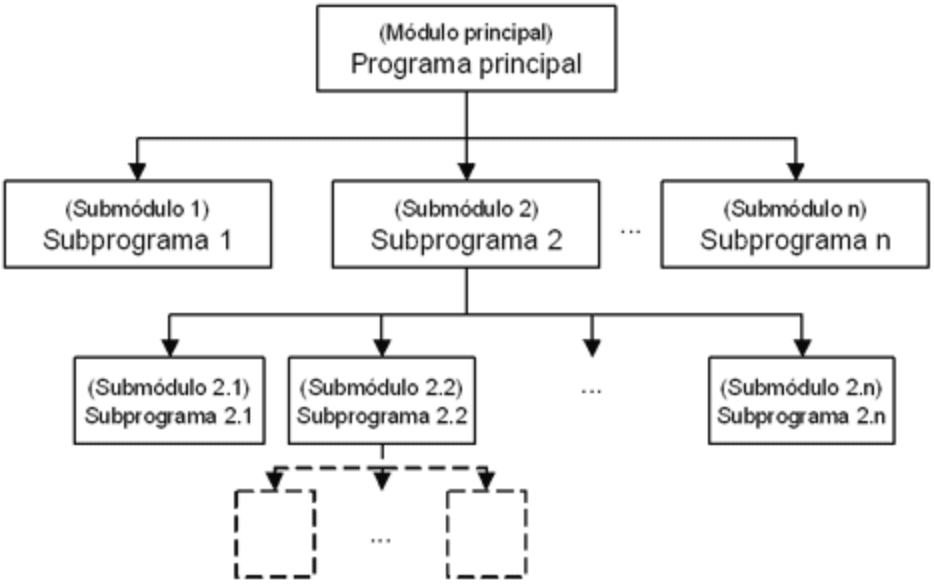


1.3. Subprogramas

Un programa PASCAL, hasta el capítulo anterior, se ha concentrado en la búsqueda de **un** algoritmo apropiado para resolver el cálculo deseado. La sección ejecutable del programa tiende a crecer considerablemente. Este estilo de programación nos lleva a programas extensos si el problema es suficientemente complejo. Los programas extensos son difíciles de corregir, modificar y mantener.

En el análisis top-down, el programador puede dividir cada tarea compleja en varias subtareas más simples, definiendo así algunos algoritmos (posteriores programas) para resolver cada una de ellas. A partir de ahora, esas subtareas serán tratadas como subprogramas.

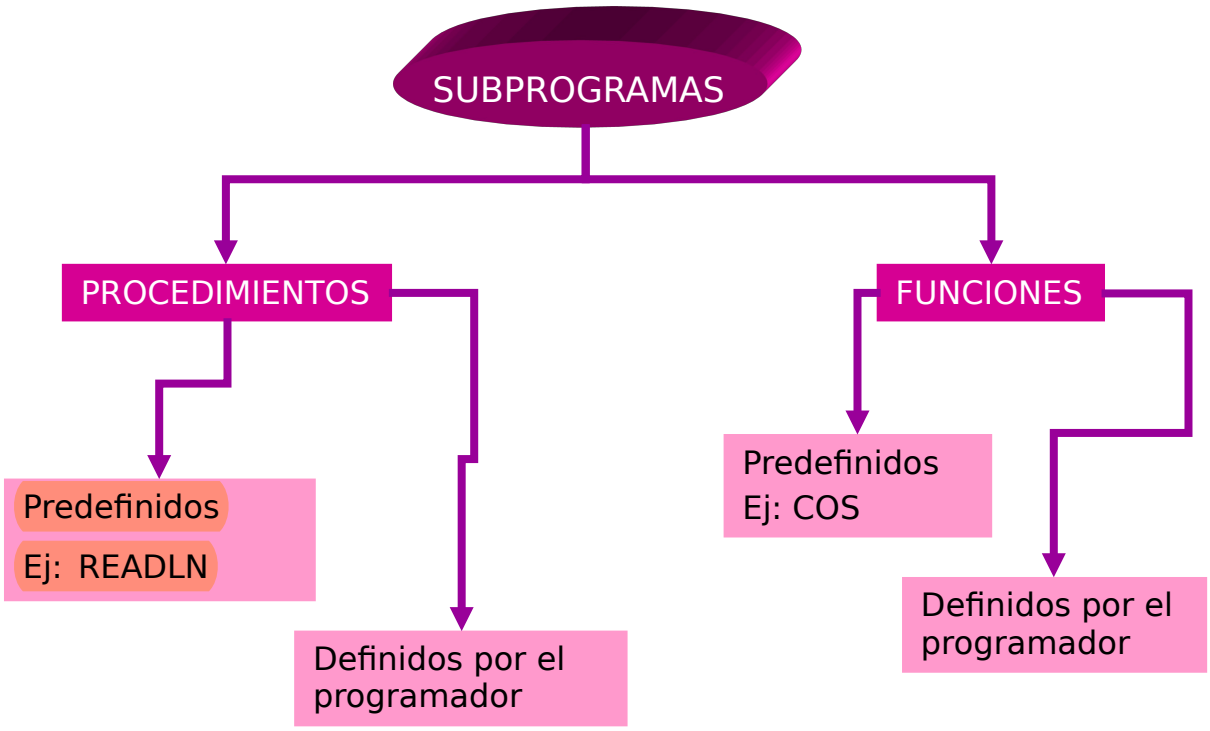
Un **subprograma** es una sección del programa (idealmente autónoma), que realiza una tarea definida, la cual puede ser llamada (activada, invocada) por otras secciones del programa, siempre que se necesite, a fin de ejecutar la tarea asignada.



1.4 Ventajas del uso de subprogramas

- AHORRO DE CÓDIGO: tareas que se repiten pueden ser escritas en PASCAL sólo una vez en un subprograma, luego se le llama tantas veces como sea necesario.
- MODULARIDAD: los problemas son divididos en subproblemas (estrategia top-down) en forma conveniente; luego cada uno de ellos pueden ser llevados a subprogramas.
- BUEN MANTENIMIENTO: los subprogramas pueden ser corregidos y modificados fácilmente. Es absolutamente conveniente que la comunicación entre el programa principal y el subprograma sea SÓLO a través de sus parámetros.

1.5 Tipos de subprogramas



Ejemplos:

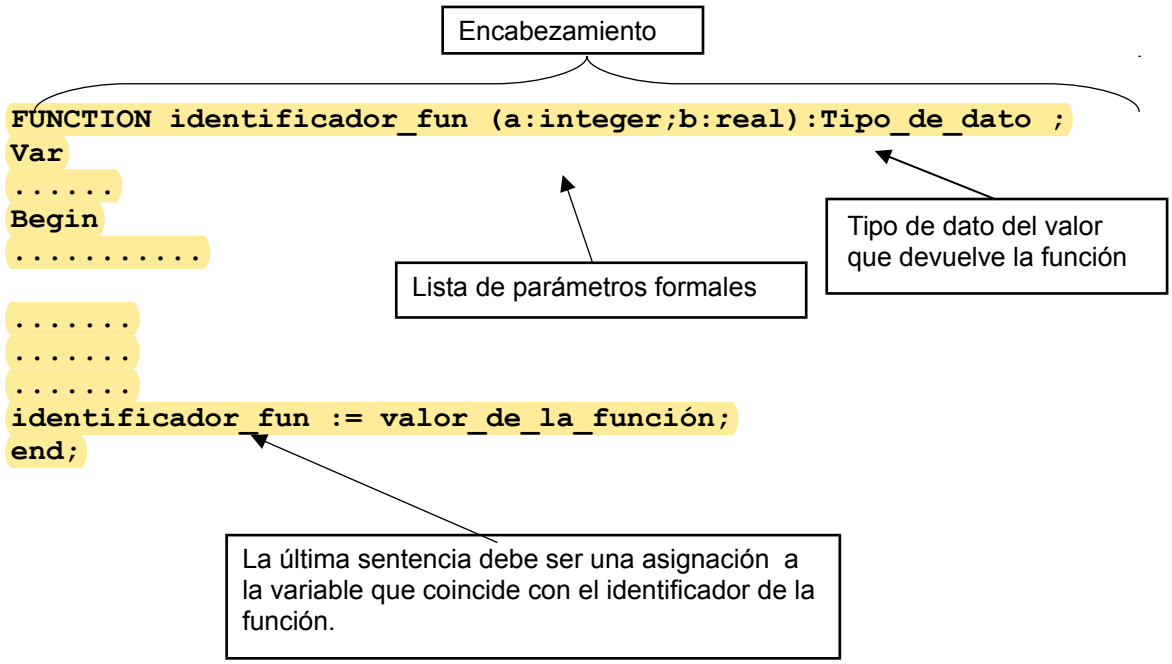
```
clrscr; {procedimiento que borra la pantalla}

writeln(A); {writeln es un procedimiento}
B:=Sqr(C*D);   {Sqr es una función predefinida}
```

2. Funciones

Una función es un subprograma que recibe como parámetros los datos que requiere para cumplir su objetivo y devuelve un resultado al programa que es el *valor* de esa función[3].
Una función siempre retorna un sólo valor.

2.1 Declaración de funciones



En el encabezamiento además del identificador y los parámetros de entrada se debe indicar el tipo del resultado que devolverá la función.
En el cuerpo de la función debe regresarse el valor de la función de acuerdo a la estructura indicada anteriormente.
Este valor se conoce como valor de retorno o valor de regreso de la función, si se omite el compilador no marcará error alguno, pero puede regresar "basura" o información sin sentido. Pueden incluirse varias sentencias que regresen un valor, pero el valor regresado será la última asignación hecha durante el flujo de la función.

Los parámetros de entrada se denominan ***parámetros formales*** son variables de determinado tipo cuyo objetivo es especificar la comunicación del subprograma con el programa (el cual generalmente pasa a llamarse programa principal u otra subprograma). Estos parámetros (en otros lenguajes se los llama "mudos") no actúan como variables comunes, i.e., no se les asignan valores durante su definición.

2.2 Llamado a funciones

El llamado a la función no genera una sentencia. Este debe hacerse como parte de una expresión, es decir, debe figurar a la derecha del signo de asignación, en sentencia write o en una expresión.

[3] Lage F., Cataldi Z.y Zalgueiro F. (2008). Fundamentos de Algoritmos y Programación. Editorial Nueva Librería.

Como se aprecia en el esquema a continuación, la función replica el formato de un programa con la excepción de la palabra clave **function** en lugar de **program,** seguido por la lista de parámetros formales entre paréntesis**.**

```
PROGRAM XX;
Var
    a:real;
    b1,b2:integer;

{Declaracion de una función}

FUNCTION Ejemplo(x,y:integer):real ;
Begin
    ...........
     Ejemplo:= .........;
End;

Begin {p.p.}
    .......
    a:=Ejemplo(b1,b2)+12.04;
     ........
End.
```

La invocación (en una expresión) activa la función; esta es ejecutada, luego el valor guardado en Ejemplo es devuelto al p.p.

Los ***parámetros actuales*** son expresiones (entre ellas variables y constantes) que contienen la información necesaria para la ejecución del procedimiento. Estos parámetros se activan cuando la función es invocada para su ejecución.

E1) Este programa contiene una función que permite calcular el Máximo Común Divisor entre dos números.

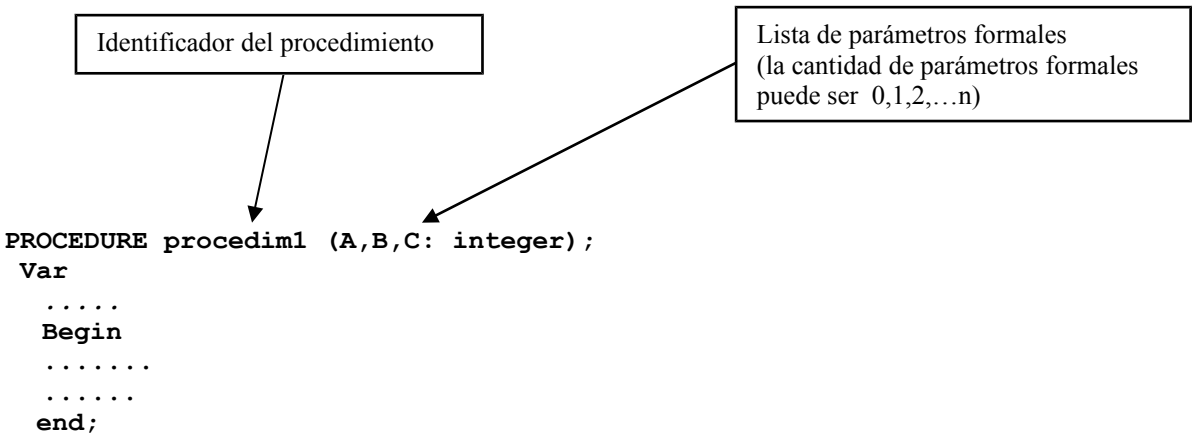
```
program func1;
var
  x,y,r:integer;

Function mcd(a,b:integer):integer;
 begin
  while a<>b do
     begin
     if a>b then
        a:=a-b
     else
        b:=b-a;
     end;
   mcd:=a;
 end;{Fin de la función}
begin{P.P}
 writeln('Ingrese dos numeros');
 readln(x,y);
 r:=mcd(x,y);
 writeln('El mcd es :',r);
end.
```

3. Procedimientos

Un procedimiento es un subprograma que puede recibir cero, uno o más parámetros y devolver al módulo que lo invoca: ninguno, uno o más valores.

3.1 Declaración de los procedimientos



El procedimiento replica el formato de un programa con la excepción de la palabra clave **procedure** en lugar de **program,** seguido por la lista de parámetros formales entre paréntesis**.**

3.2 Llamado a los procedimientos

Para lograr que un subprograma se ejecute, este debe ser llamado (invocado) desde la sección ejecutable del programa. Para invocarlo se debe escribir el identificador del procedimiento seguido de parámetros actuales entre paréntesis.

```
PROGRAM principal;

   CONST
      {Definición de constantes}

   TYPE
      {Declaración de tipos}

   VAR
      {Declaración de variables}
       A,B,C:integer;

   PROCEDURE procedim1 (A,B,C:integer);

     ! Sección de declaraciones del procedimiento
     ............
     ! Sección ejecutable del procedimiento
      Begin
      .......
      ......
     end;

   BEGIN

      {Sentencias ejecutables del programa}

         Procedim1(A,B,C);

         . . . . . . .

   END.
```

Segundo paso: Luego de su ejecución, el procedimiento devuelve los resultados a traves de sus parámetros.

Primer paso: Luego de su invocación, el control de ejecución pasa al procedimiento

Parámetros actuales

Cuando se lo llama, el procedimiento es ejecutado

Cuando, durante la ejecución del programa Principal, el procedimiento es llamado, el control de ejecución pasa al procedimiento. Este es ejecutado completamente, resolviendo un problema específico, enviando la solución al programa principal a través de los parámetros. Inmediatamente, el control de ejecución pasa a la sección ejecutable del programa, específicamente a la sentencia siguiente de la invocación del procedimiento.

E2) Dibujar, N veces, el siguiente esquema :
++++++++++
**********,

El usuario puede elegir los caracteres y las cantidades a dibujar de cada carácter (el programa realiza dibujos utilizando parámetros).

```
PROGRAM trama;
Var
I,veces:integer;
caract1,caract2:char;
cant1,cant2:integer;
```

```
PROCEDURE dibujo(caracter:char;cant:integer);
Var
  j:integer;
BEGIN
    For j:=1 to cant do
          Write(caracter);
    Writeln
END;

BEGIN   {p.p.}
   Writeln('Ingrese las veces que repite los dibujos');
   Readln(veces);
   Write('Cant. de caracteres para el primer dibujo');
   Readln(cant1);
   Write('carácter del primer dibujo');
   Readln(caract1);
   Write('Cant. de caracteres para el segundo dibujo');
   Readln(cant2);
   Write('carácter del segundo dibujo');
   Readln(caract2);
    For i:=1 to veces do
         Begin
           Dibujo(caract1,cant1);
           Dibujo(caract2,cant2)
         End;
 END.
```

4. Ámbito de las variables

- Definimos como ámbito de un objeto (variable, tipo, etc.) a la esfera de acción de dicho objeto. El ámbito puede ser *global* o *local*.

- En la declaración de un programa (*programa principal*) encontramos la sección de definición de constantes, la sección de definición de tipos, la sección de declaración de variables, procedimientos y funciones que son *globales* al programa. Esto significa que son válidos en todo el programa.

- En la declaración de un procedimiento tenemos identificadores de parámetros (formales), la sección de definición de constantes, la sección de definición de tipos, la sección de declaración de variables, procedimientos y funciones que son **locales** al procedimiento declarado, i.e., *no se les conoce* fuera de él. Queda claro que los objetos globales siguen valiendo dentro del procedimiento**.**

- Cuando hay una variable global y una local que tienen el **mismo nombre**, dentro del subprograma la variable global no es visible. Esto no significa que la variable global no exista; simplemente no es visible: todas las ocurrencias del nombre dentro del subprograma se refieren a la variable local.

Veamos el siguiente ejemplo:

```
Program func3;
  Var
      n1,n2,res1,res2:integer;
Function suma(limite:integer):integer;
   Var
        numero,acumula:integer;
   Begin
        acumula:=0;
        For numero:=1  to limite do
             acumula:=acumula+numero;
        suma:=acumula
  End;
Begin
        Readln(n1,n2);
        res1:=suma(n1);
        res2:=suma(n2) div 4+res1;
End.
```

Bloque	Identificador	Significado de cada identificador
P.P	n1,n2,res1,res2	Variables globales
	Numero,acumula	Variables locales
	limite	parámetro

Si bien la implementación del lenguaje permite modificar las variables globales en cualquier punto del programa, se considera una **mala práctica** de programación extraer resultados de los procedimientos a través de ellas. Recordar que el mecanismo de comunicación entre el programa principal y los procedimientos debe hacerse SÓLO a través de los parámetros. Siguiendo esta recomendación, se facilita la corrección de los subprogramas evitando los efectos secundarios.

5. Pasaje de parámetros

El mecanismo de comunicación entre el programa principal y los procedimientos consiste en el pasaje de datos a través **de parámetros**.

- Para que exista comunicación entre programa principal y los procedimientos se deben usar parámetros (en caso contrario no hay comunicación o será considerada errónea, por ejemplo cuando se utilicen variables globales).

- Como ya se analizó, los parámetros que figuran en la declaración del procedimiento, se denominan **parámetros formales**. Debe indicarse el identificador y tipo del parámetro.

- Los parámetros que figuran en la sentencia de llamada al procedimiento se denominan **parámetros actuales.** Sólo debe indicarse su identificador (o expresión), que debe figurar en la sección de variables globales del programa principal.

- La cantidad entre parámetros actuales y formales debe coincidir. Deben ser compatibles en tipo. La correspondencia entre actuales y formales se determina por la posición en su respectiva lista.

- Los nombres de los parámetros formales y actuales que se correspondan **pueden coincidir** entre sí. Si el programador lo desea puede darles nombres distintos. Por ejemplo:

En el Curso de Computación de la Plataforma Moodle se presenta un material referido al tema "Pasaje de Parámetros"

Veremos el *pasaje de parámetros por valor* y el *pasaje de parámetros por variable.*

5.1 Pasaje de parámetros por valor

Veamos el siguiente ejemplo:

```
PROGRAM EjemploValor;

Var
    x:integer;
PROCEDURE cambio(y:integer);{y es un parámetro formal por valor}
      Begin
         y:=1
      End;
BEGIN
     x:=0;
     Cambio(x);{x es un parámetro actual por valor}
     Writeln('El valor de X es ',x)
END.
```

Las características del pasaje de parámetros por valor son:

1.-Se establece una comunicación unidireccional (del programa principal hacia el procedimiento).
2.- Al parámetro formal se le asigna memoria y se le copia el valor del parámetro actual y se corta la comunicación entre programa principal y el procedimiento.
3.- Si se modifica el valor del parámetro formal dentro del procedimiento, el valor del parámetro actual NO CAMBIA.
4.- Si bien el parámetro formal debe ser una variable, el parámetro actual puede ser una expresión.

Sintaxis de la cabecera de un procedimiento con parámetros valor en la sección de declaración y del llamado:

```
Procedure Ejemplo(a,b:integer;zeta:real);
```

Los parámetros formales por valor se declaran en una lista similar a la *declaración de variables de la sección VAR*

```
Ejemplo(alt,7,cos(t)*sqrt(z));
```

Los parámetros actuales por valor se denotan usando expresiones del mismo tipo que su correspondiente parámetro formal

A continuación se muestra el pasaje de parámetros en el procedimiento Ejemplo:

```
PROCEDURE Ejemplo(A,B:integer; zeta:real); {cabecera de la declaración}

          Ejemplo( A, 7 , Cos(t)*sqrt(Z) );  {sentencia de llamado}
```

5.2 Pasaje de parámetros por variable o referencia

Veamos el siguiente ejemplo:

Para declarar que un parámetro es pasado por referencia o variable, debe estar precedido por la palabra reservada `var`

```
PROGRAM EjemploVariable;
Var
    x:integer;
procedure cambio(var y:integer);
 begin
    y:=1;
 end;
begin {programa principal}
    x:=0;
    cambio(x);
    writeln('el valor de x es: ',x);
end.
```

Las características del pasaje de parámetros por variable son:

1.- Se establece una comunicación bidireccional
2.- Cada vez que se modifica el parámetro formal, se modifica el parámetro actual. De esta forma, se pueden intercambiar información datos. El programa principal le envía datos al procedimiento y el procedimiento le envía resultados al programa principal.
3.- No hay copia de valores. Se establece una referencia entre ellos, es decir, el parámetro actual le pasa la dirección donde él se encuentra, al parámetro formal.
4.- Los parámetros formales y actuales deben ser variables.

Sintaxis de la cabecera de un procedimiento con parámetros variable y del llamado:

```
Procedure Ejemplo(var a,b:integer;var zeta:real);
```

Los parámetros formales por variable se declaran en una lista similar a la declaración de variables de la sección VAR, pero debe escribirse la palabra reservada ***var*** *antecediendo cada declaración*

```
Ejemplo(a,b,w);
```

Los parámetros actuales por variable se denotan usando variables (solamente) del mismo tipo que su correspondiente parámetro formal. Como ya se presentó más arriba, el nombre del parámetro puede coincidir o no.

E3) A continuación se presenta un ejemplo completo

```
Program Completo;
Var
   dato1:integer;
   dato2,dato3:real;
Procedure ejem1(param1:integer; VAR param2,param3:real);
    Var
      V1,V2:integer;
    Const
       C1=1.5;   C2=2.5;
    Begin
        Writeln('Ingrese un dato entero');
        Readln(V1);
        V2:=V1*param1;
        Param2:=param2/C1;
        Param3:=V2*C2;
   End;
Begin  {P.P.}
    Writeln('Ingrese un dato entero y uno real');
    Readln(dato1,dato2);
    Ejem1(2*dato1,dato2,dato3);
    Writeln('El resultado es:',dato1,dato3:4:2,'y',dato2:4:2);
    Readln
End.
```

6. Anidamiento de subprogramas

Dentro de cada procedimiento puede declararse otros procedimientos , dando lugar al **anidamiento de subprogramas**.

- El bloque del programa principal es el nivel 1, el menor nivel del programa: es la base sobre la que se pueden construir más niveles.
- Cuando se agregan procedimientos (y funciones) al programa creamos un nivel 2. En este caso los procedimientos están anidados en el programa.
- El anidamiento de procedimientos dentro de otros procedimientos permite añadir mas niveles al programa.
- El anidamiento de procedimientos crea "privacidad" entre los procedimientos.
- Un procedimiento que está anidado dentro de otro es privado a ese procedimiento.
- El mantenimiento de procedimientos privados puede disminuir los errores de programación al limitar el uso de procedimientos a secciones específicas del programa.

A continuación vemos un programa simple donde se aprecia el anidamiento de procedimientos:

E5)

```
Program anidamiento;
Var
   A,b,c:integer; {variables globales}

Procedure Primero(VAR pe,ju:integer);
```

```
Var
  D:integer; e:real;{var. locales a Primero y globales a Segundo}

     Procedure Segundo(Pi:integer;VAR jo:real);
          Var
             F:real;
          Begin{seccion ejecutable de Segundo}
             F:=Pi+0.28;
             Jo:= F/23.7+345.6
          End;
  Begin {sección ejecutable de primer}
     Readln(D);
     Segundo(D+2,e);
     Writeln(e);
  End;

Procedure tercero(VAR lu:integer);
   Var
     H:real;k:integer;
   Procedure Cuarto(ne:integer);
          Var
            I:real;
          Begin {sección ejecutable de  cuarto}
            I:=sqrt(ne)*(ne-2.34);
            Writeln(i)
          End;
   Begin {sección ejecutable de tercero}
       Readln(h,k);
       If h<lu then
           cuarto(k)
       else
           cuarto(2*k);
   End;

Begin {seccion ejecutable del p.p.}
  Primero(a,b);
  Readln(c);
  Tercero( c)
End.
```

Recordar que:

- No se puede llamar a los procedimientos antes de haberlos declarado.
- La **regla de precedencia** se basa en el principio de que una idea compleja se construye mejor a partir de ideas simples. Es decir, un programa se construye a partir de pequeños y sencillos procedimientos que se combinan para formar procedimientos que se hacen cada vez mas complejos.

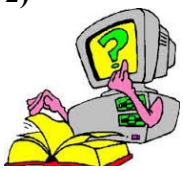
7. Ejercicios

1) Considerar la función estación escrita abajo. Escribir un programa que lea la fecha (dia y mes) de nacimiento de N personas, y mediante el uso de la función determine cuántas de ellas nacieron en verano.

```
Function Estacion(d,m:byte):string;
Begin
  Case m of
    1,2:Estacion:='verano';
    4,5:Estacion:='otoño';
    7,8:Estacion:='invierno';
```

```
     10,11:Estacion:='primavera';
     3:If d<=20 then
          Estacion:='verano'
       else
          Estacion:='otoño';
     6:If d<=20 then
          Estacion:='otoño'
       else
          Estacion:='invierno';
     9:if d<=20 then
          Estacion:='invierno'
       else
          Estacion:='primavera';
     12:if d<=20 then
          Estacion:='primavera'
        else
          Estacion:='verano';
   else
      Estacion:='fecha incorrecta'
   end;
end;
```

2)



a) Investigar la utilidad y el tipo de salida de las funciones predefinidas de FreePascal: *round, trunc, int, frac*. Escribir un programa que permita probarlas aplicándolas a los valores -4.537 y 4.537

b) Una librería trabaja con una tarjeta de puntos para fidelizar a sus clientes. Por cada compra que realizan los clientes, se ingresa la siguiente información:

- Nro. de Tarjeta
- Importe de la Compra actual
- Puntos Acumulados (antes de esta compra)
- ¿Acumula puntos o Canjea puntos?

Con cada compra el cliente puede decidir acumular puntos (10 puntos cada $100 de la compra) o utilizar puntos para pagar el total de la compra (50 puntos cada $200)
Aclaración: no se consideran casos intermedios, es decir $120 de compra NO equivalen a 12 puntos acumulados sino sólo a 10; y en el caso de canje de puntos, compras entre 0 y $200 requieren entregar 50 puntos, superiores a $200 y hasta $400 requieren 100 puntos y así sucesivamente.
NOTA : una compra no se puede pagar con puntos y plata, o sólo puntos o sólo dinero

Se pide desarrollar un programa Pascal, correctamente indentado y eficiente, que ingrese los datos de M compras y resuelva:
a.- Indicar cuántos clientes han intentado canjear puntos y no han podido (pues no tenían puntos suficientes para cubrir el valor de la compra)
b.- Para cada tarjeta, la cantidad de puntos acumulados luego de esta compra (haya o no acumulado en esta compra)

3) Identificar los errores cometidos en el siguiente programa que utiliza la función FF:

```
   Program Corto;
      Var
         a,b,c,d:integer;
      Function FF(x,y:integer):integer;
      Var
         h:integer;
```

```
Begin
   h:=x+y;
   FF:=(x*y)mod h;
End;

Begin{PP}
   Readln(a,b);
   FF(a,b);
   Writeln(FF);
   c:=2*FF(b,2-a);
   d:=FF(a/2,b);
   d:=FF(5,3);
   writeln(FF(a,b));
End.
```

4) a) Construir una función que, invocada como EXPO(x,n), devuelva el valor de x^n, donde x es un número real <u>no nulo</u> y n es un número entero. (Observación: tener en cuenta el signo de la base y del exponente)

b) Escribir un programa que, invocando la función construida, calcule y muestre por pantalla los 5 primeros términos de la sucesión cuyo término general es $a_n = \left(\frac{3}{2}\right)^{n-3} + (-2)^{n+1}$

5) a) Investigue el modo de uso de la función predefinida *Random* y el procedimiento *Randomize*

b) Escribir una función que devuelva un número aleatorio ***entero*** en un rango a elección del operador. Utilizarla en un programa que genere N números aleatorios en un rango dado por el usuario.

c) Escribir una función que devuelva un número aleatorio ***real*** en un rango a elección del operador. Utilizarla en un programa que genere N números aleatorios en un intervalo real determinado por el usuario.

6) a) Desarrollar una función que, dado un número entero no negativo, obtenga su factorial. Escribir un programa que calcule y muestre el factorial de N números elegidos por el usuario.

b) Escribir un programa que calcule el número combinatorio $\binom{m}{k} = \frac{m!}{k!(m-k)!}$ de dos valores enteros no negativos m y k, siendo m ≥ k.

7) a) Desarrollar una función que devuelva True si la primera fecha es anterior a la segunda, y False en caso contrario. Las dos fechas están expresadas como ternas de números enteros Año1-Mes1-Dia1 y Año2-Mes2-Dia2.

b) Escribir un programa que lea los nombres y las fechas de nacimiento de tres personas, expresadas como se indicó en el inciso anterior, y determine e informe el nombre de la más joven.

c) En un archivo Fechas.txt se han almacenado una cantidad no conocida de fechas, una por línea con el formato aaaa mm dd. Escribir un programa que determine si las fechas del archivo están o no ordenadas cronológicamente.

8) Dado el siguiente programa:

```
Program Simple;
    Const
        c=4;
    Var
        A,B,D:Integer;
        G:boolean;
```

```
Procedure SALUDO(k:integer);
    Begin
       Writeln('OPERAREMOS CON ', k);
       Writeln('**********************');
    End;
Procedure SUMA(X:Integer;Var Res:Integer; Var gr: boolean);
Var
Aux: integer;
    Begin
       gr:=false;
       Res:=0;
       Aux:= sqr(X);
       While X>0 Do
        Begin
           Res:=Res+X;
           X:=X-1;
        End;

        gr:=res>aux;
    End;
Procedure LIO(var g:integer);
    Var
       u:integer;
    Begin
       writeln('valor?');
       readln(u);
       g:=g+u;
    End;

    Begin
       Readln(A);
       SALUDO(A);
       D:=Abs(A);
       SUMA(D,B,G);
       Writeln('Resultados = ',B);
       If G then
          Writeln('la sumatoria supera al cuadrado de ', D);
       Writeln('Ahora D vale ',D);
       LIO(A);
       Writeln('A = ',A);
    End.
```

a) Identificar parámetros formales y actuales y parámetros que se pasan por valor o variable.
b) El contenido de la variable D ¿cambia luego de llamar al procedimiento SUMA? Justificar.
c) El contenido de la variable A ¿cambia luego de llamar al procedimiento LIO? Justificar.
d) Justificar cuáles de los siguientes llamados al procedimiento SUMA son correctos:

i) SUMA(A, B,G); ii) SUMA(C, B, G); iii) SUMA(A, C, G);
iv) SUMA(3, B, true); v) SUMA(3, 6, G); vi) SUMA(X, Res, gr)

9) Considerar el siguiente programa:

```
PROGRAM ENSAYO;
...
Procedure Prueba(L:integer; var M:integer);
 ....
 Begin
    ....
 End;
Begin{PP}
 ....
 Prueba(C,D);
    ....
End.
```

Indicar V o F justificando su respuesta:

a) Cuando se invoca el procedimiento Prueba se crea una posición de almacenamiento para L
b) Cuando se invoca el procedimiento Prueba no se reserva ninguna posición de memoria para almacenar el valor de M.
c) Cualquier cambio de M dentro del procedimiento no constituye cambio para D.
d) L puede usarse para devolver un valor al programa principal.
e) C y D deben ser variables enteras

10) Dado el siguiente segmento de programa:

```
Program EJEMPLO;
  var
     x : integer;
     y : real;
     z : char;
     ................
  procedure prueba(letra:char; i:integer; j:real);
     begin
        ..................
        ..................
     end;
  begin
     .............
     .............
  end.
```

¿Cuál de las siguientes invocaciones al procedimiento es correcta? Justificar las incorrectas.

a) prueba (y,x,z);
b) prueba (letra, i, j);
c) prueba (z, x, y);
d) prueba;
e) h:= prueba(z, x, y);

11)

a) Completar las líneas punteadas de modo que el programa:

- lea las componentes real e imaginaria de dos números complejos, y luego
- ofrezca un menú para resolver las cuatro operaciones básicas e imprima por pantalla los resultados.

El menú debe reiterarse hasta que el usuario decida salir.

NOTA: la cantidad de líneas NO indica cantidad de sentencias requeridas.

```
Program nume_complejos;
   var
      PR1,PI1,PR2,PI2,PR3,PI3:real;
      Op:char;
      Leyo:boolean;

Procedure leer (var re,im:real);
   Begin
      Writeln('Ingrese la componente real');  Readln(re);
      Writeln('Ingrese la componente imaginaria');  Readln(im);
   End;
Procedure escribir (re, im: real);
   Begin
      write(re:5:2);
      If im >=0 then
         write('+');
      writeln(im:5:2, 'i');
   end;
Procedure Suma(R1,I1,R2,I2:real; var R3,I3:real);
```

```
   Begin
      R3:=R1+R2;   I3:=I1+I2;
   End;
Procedure Resta(R1,I1,R2,I2:real; var R3,I3:real);
   Begin
      R3:=R1-R2;   I3:=I1-I2;
   End;
Procedure Producto(R1,I1,R2,I2:real; var R3,I3:real);
   Begin
      R3:=R1*R2-I1*I2;     I3:=(I1*R2+R1*I2)/aux;
   End;
Procedure Division(R1,I1,R2,I2:real; var R3,I3:real);
   Var
      aux:real;
   Begin
      aux:=sqr(R2)+sqr(I2);
      R3:=(R1*R2+I1*I2)/aux;  I3:=(I1*R2+R1*I2)/aux;
   End;
Procedure Menu(var Op:char);
   Begin
      Writeln('Ingresar números: i') ;
      Writeln ('Seleccionar operación a realizar: +  -  *  /');
      Writeln ('para finalizar ingrese F');
      Readln(Op);
   End;

Begin {PP}
   Leyo:=false;
   Repeat
      Menu(Op);
      If Op<>'F' then
         ..............................................................
         ..............................................................
         ..............................................................
         ..............................................................


      Else
         ..............................................................
         ..............................................................
         ..............................................................
         ..............................................................
         ..............................................................
   until Op='F';
   readln;
end.
```

b) Identificar parámetros formales y actuales y parámetros que se pasan por valor o variable.

12) Escribir un programa que lea dos valores reales **h** y **r**, correspondientes a la altura de un cilindro circular recto y al radio del mismo, respectivamente. Luego, <u>mediante un menú</u> implementado a través de un procedimiento, ofrezca las siguientes opciones de cálculo (cada una resuelta a través de una función) y muestre el resultado con dos decimales:

a) Perímetro de la circunferencia de la base ($2 \pi r$)
b) Superficie de la base del cilindro (πr^2)
c) Volumen del cilindro (Sup. de la base * h)
d) Superficie lateral del cilindro (Perímetro de la base *h)

13) a) Escribir un procedimiento que, dadas las coordenadas de 2 puntos en el plano, obtenga la pendiente y la ordenada al origen de la recta que los contiene (considerar que los puntos tienen distinta abscisa).

b) Para cada uno de los siguientes problemas, escribir un programa que los resuelva invocando al procedimiento anterior:

b.1) Para un punto dado (x_0,y_0) y una serie de pares de puntos determinar qué porcentaje de los pares están alineados con (x_0,y_0).

b.2) Dados 4 puntos no alineados indicar si determinan un paralelogramo (cuadrilátero con 2 pares de lados paralelos)

b.3) Dados los puntos del plano P_1, P_2, P_3 y P_4, considerar las rectas P_1P_2 y P_3P_4 y determinar si éstas son paralelas o secantes. En éste último caso informar si son o no perpendiculares.

14) El siguiente procedimiento calcula las raíces reales de una ecuación cuadrática:

```
Procedure Calcula (A,B,C: real; Var x1,x2:real);
  Begin
     X1:=(-B+sqrt(sqr(b)-4*A*C))/(2*A);
     X2:=(-B-sqrt(sqr(b)-4*A*C))/(2*A);
  End;
```

a) ¿Es posible invocarlo si se desea hallar las raíces de la ecuación $2x^2+3x+5=0$?

b) Escribir una función booleana que, dados los tres coeficientes de una ecuación cuadrática, determine si tiene o no raíces reales.

c) Escribir el programa que analice, mediante la función del inciso **b)**, las distintas posibilidades respecto a las raíces en una ecuación de 2º grado y, en caso de que existan raíces reales, las calcule y muestre con dos decimales.

15) Escribir un programa modularizado que lea los valores de la pendiente **m** y la ordenada al origen **b** de una recta R. Luego debe presentar un menú que ofrezca las siguientes opciones:

a) Dada una serie de puntos del plano, cuya cantidad no se conoce previamente, calcular y mostrar por pantalla la cantidad de puntos que pertenecen a R.

b) Dado un punto de coordenadas (h, k), escribir por pantalla la ecuación de la recta paralela a R que contiene al punto (h, k).

c) Obtener el ángulo que forma la recta R con el semieje x positivo, expresándolo en grados sexagesimales.

El menú debe reiterarse hasta que el usuario decida salir.

16) Escribir un programa modularizado que simule el juego de dados "Obligada" entre dos jugadores e informe cuál de ellos es el ganador. Las reglas del juego son: hay 6 rondas en las que cada jugador, por turno, arroja los 5 dados. En la primera ronda, se suman los 'unos' obtenidos, en la segunda ronda se suman los 'dos' obtenidos, y así sucesivamente hasta los 'seis'. Gana el jugador que obtuvo el máximo puntaje.

17) Escribir un programa modularizado que simule el siguiente juego de dados entre dos jugadores e informe cuál de ellos es el ganador. Las reglas del juego son:

- Cada jugador comienza su turno arrojando los 5 dados.
- Los dados que suman puntaje son el as (100 puntos) y el 5 (50 puntos).
- La tirada se considera válida si se obtienen dados que suman puntaje.
- Si la tirada es válida, el jugador puede realizar una nueva tirada con los dados que NO resultaron ases ó cincos. Puede tirar hasta tres veces, siempre que le queden dados sin puntaje.
- Si el jugador NO obtiene dados que suman puntaje, se queda sin dados, o ya realizó 3 tiradas, se pasa el turno al otro jugador.
- El juego termina cuando uno o ambos jugadores alcanza los 1.000 puntos. Gana el jugador con mayor puntaje.

Ejemplo:

JUGADOR	DADOS EN LA TIRADA	PUNTAJE POR TIRADA	PUNTAJE ACUMULADO
A	Primera tirada: 1 2 2 3 5 Segunda tirada: 3 4 1 Tercera tirada: 5 2	150 100 50	150 250 300
B	Primera tirada: 1 1 2 5 3 Segunda tirada: 4 6 Tercera tirada:----------	250 0	250 250
A	Primera tirada: 2 3 4 6 2 Segunda tirada: ---------- Tercera tirada: ----------	0	300
B	Primera tirada: 1 1 4 1 2 Segunda tirada: 1 5 Tercera tirada: ----------	300 150	550 700
A	Primera tirada: 2 3 4 5 6 Segunda tirada: 3 1 6 4 Tercera tirada: 2 3 2	50 100 0	350 450 0
B	Primera tirada: 1 1 1 5 2 Segunda tirada: 5 Tercera tirada: ----------	350 50	1050 1100

Gana el jugador B